

BRIEFINGS

black hat

**Chaos by Design: The Death of Stochastic
Race Conditions in HTTP/3**

Efstratios Chatzoglou

The Fallacy of "Racing the Wire"

Traditional SPA Limit

Prior Single-Packet Attacks (SPA) over HTTP/2 focus exclusively on network-layer alignment. While elegant, these are frequently neutralized by buffering architectures inside modern edge proxies (NGINX, Envoy, HAProxy) which randomize backend execution frames.

The SSRO/Temporal Hijacking Revolution

Instead of battling transport jitter, our research introduces Server-Side Race Orchestration (SSRO) and Temporal Hijacking attacks. By hijacking HTTP/3 native primitives (QPACK – RFC 9204) and RFC 7540/9218 scheduling policies, we force the proxy's internal memory to queue and align the requests for us, eliminating network jitter entirely.

Shifting the Timing Window

The Paradigm Pivot

- Our research shifts timing control entirely from the network wire directly into the server's own local memory and parsing state-machine.
- Instead of trying to "race the wire" across unpredictable transit routes, we exploit native stream handling mechanisms within the application or reverse proxy layer.
- By forcing the server to queue and buffer requests inside its RAM before atomic execution, we achieve sub-microsecond synchronization that renders network jitter completely obsolete.

Connection vs Stream Multiplexing

QUIC Transport

Ditches underlying TCP for UDP, removing head-of-line blocking natively at the transport packet level.

Independent Streams

Every HTTP/3 request operates as an independent logical stream within a single cryptographic connection.

Parsing Scheduler

The edge proxy must process multiple concurrent (independent) streams using an internal prioritization scheduler.

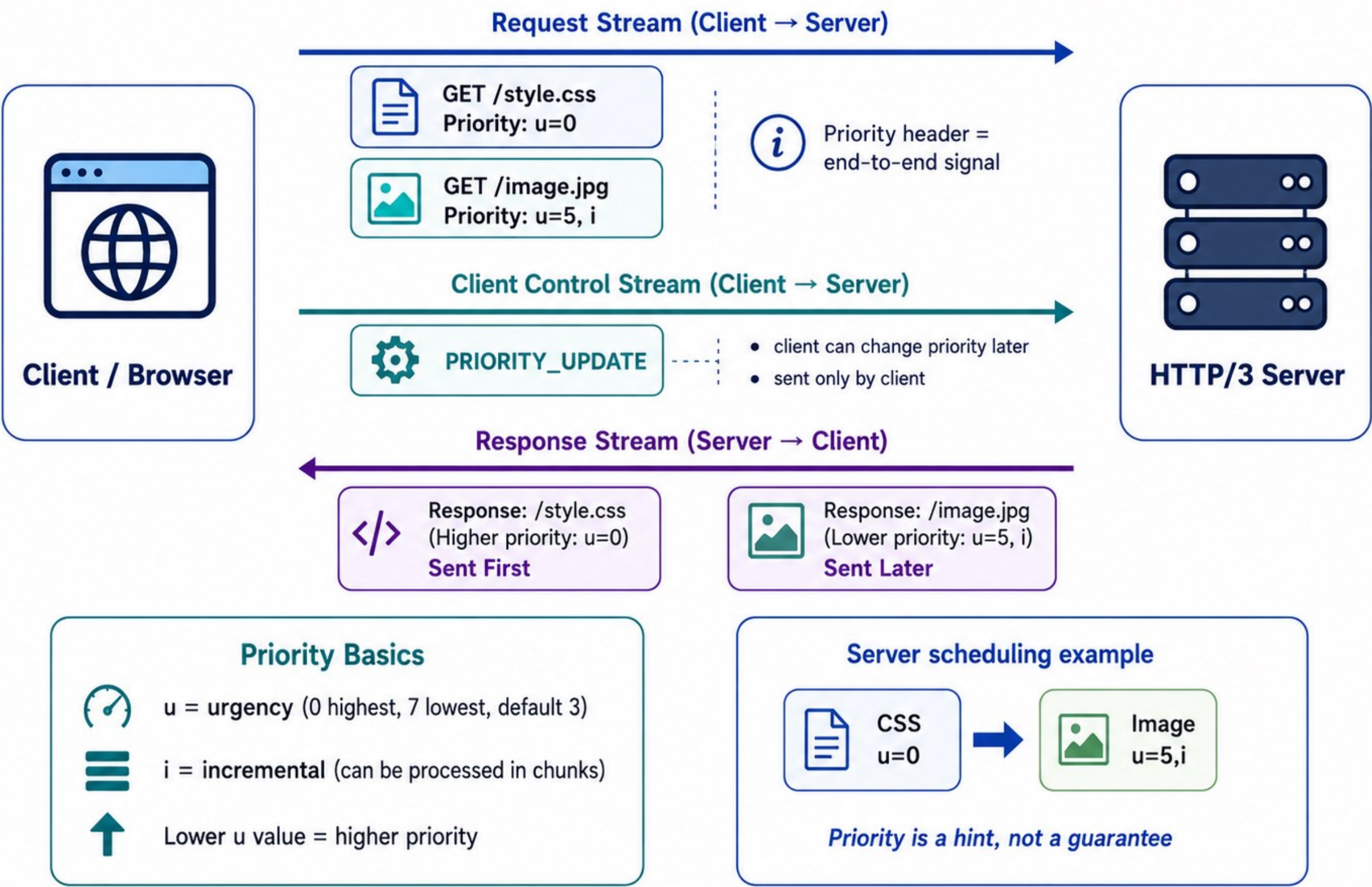
The Queue Trap

This scheduling dependency creates a local queuing mechanism in server memory, ready for exploitation.

HTTP/3 Priority (RFC 9218)

HTTP/3 Prioritization – RFC 9218

Only the essential communication flow



Temporal Hijacking: Weaponizing Urgency

Proxy Buffering Hold

Uses "header-only" transmission states to pre-allocate proxy-to-backend socket channels, letting us freeze request parsing mid-stream.

RFC 9218 Urgency

Injects extreme urgency values ($u=0$ vs $u=7$) to force late-arriving critical streams to physically leapfrog earlier ones in the memory queue.

Priority Updates

Weaponizes mid-flight `PRIORITY_UPDATE` frames to dynamically elevate streaming states after the initial handshakes (QUIC/TLS) are completed.

Legacy Tree Logic

Exploits legacy RFC 7540 weights in 70% of real-life web servers to generate complex scheduling calculations, creating synthetic delays.

Temporal Hijacking: Weaponizing Urgency

Bypassing FIFO Schedulers

Standard servers process incoming streams using a FIFO sequential queue. Temporal Hijacking leverages stream priority attributes to actively dictate the inner order of operations, forcing low-urgency commands to wait while high-urgency commands leapfrog forward.

Sub-Microsecond Control

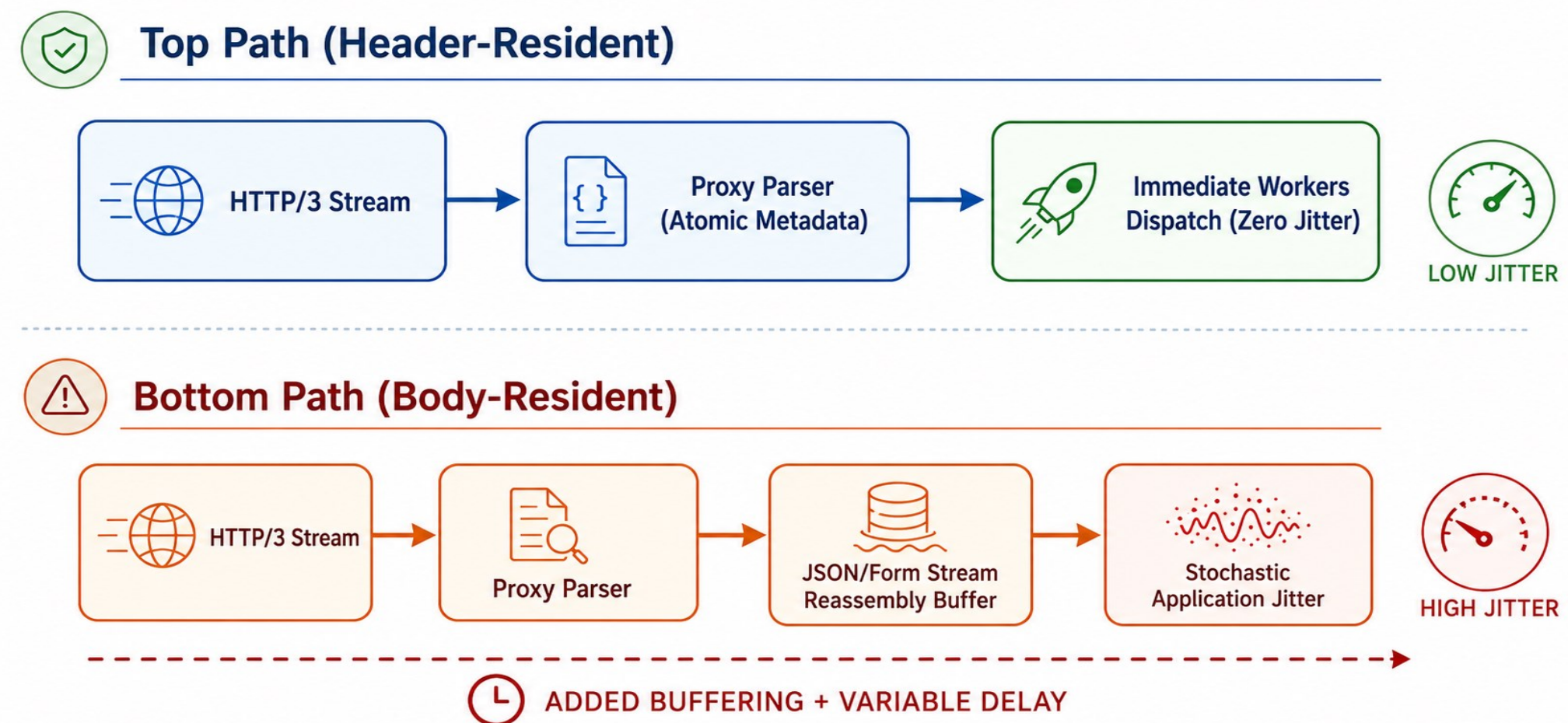
Because priority modifications occur at the connection layer prior to HTTP parsing, the scheduling calculations take place inside the proxy's native kernel space. This lets adversaries construct race states with precision, completely removing transit jitter.

Why Header-Resident Parameters Eliminate Jitter

Exploitation is significantly more reliable when vulnerable parameters are header-resident rather than body-resident.

The Mechanism: Headers are parsed as atomic blocks by the proxy edge. The internal scheduler dispatches them instantly.

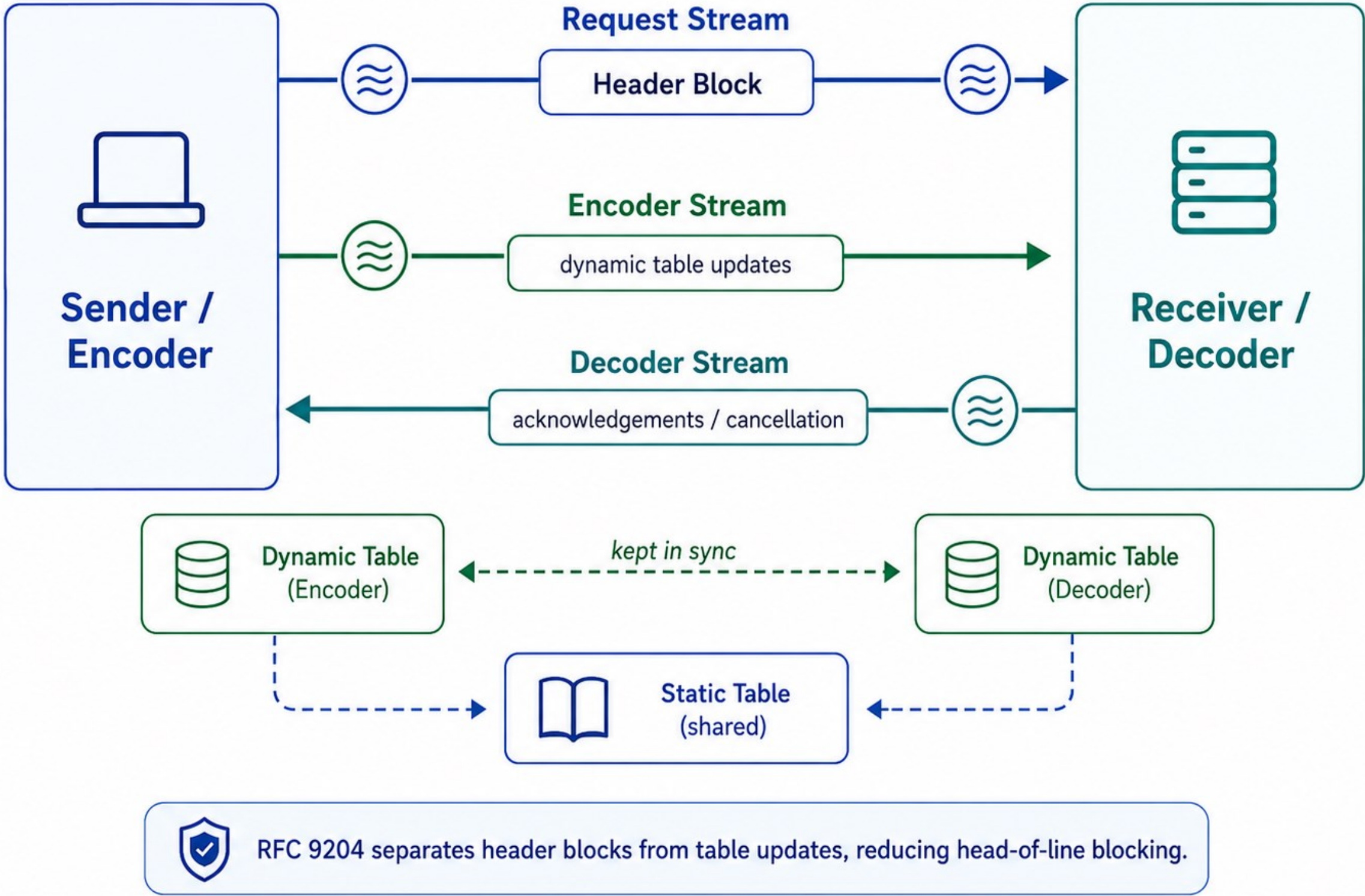
The Vulnerability: If state-changing variables (e.g., user_id, amount) reside in the headers rather than a JSON body, transport-layer synchronization maps perfectly to application-layer execution, completely neutralizing application-layer parsing jitter.



HTTP/3 QPACK (RFC 9204)

QPACK in HTTP/3 (RFC 9204)

Simple architecture overview



Weaponized Theory: HPACK vs. QPACK

HPACK (HTTP/2)

Designed for **TCP (RFC 7541)**. Requires strict, in-order delivery at the transport layer. A single lost packet stalls the entire compression state (HoL blocking).

Transport: SEQUENTIAL

QPACK (HTTP/3)

Designed for **QUIC/UDP (RFC 9204)**. Handles out-of-order delivery. Introduces asynchronous header decoding using dynamic table state tracking.

Transport: ASYNCHRONOUS

***The Pivot:** RFC 9204 was designed to *prevent* HoL blocking. We turn it upside down to create HoL blocking on purpose, pausing requests in the proxy's memory.

The "Waiting Room" Mechanic

Required Insert Count (RIC)

If a header block references a dynamic table entry the decoder hasn't acknowledged, the stream **must block**. The request sits in the proxy memory, fully parsed but undispached.

$RIC > InsertCount\ decoder \rightarrow STREAM_{BLOCKED}$

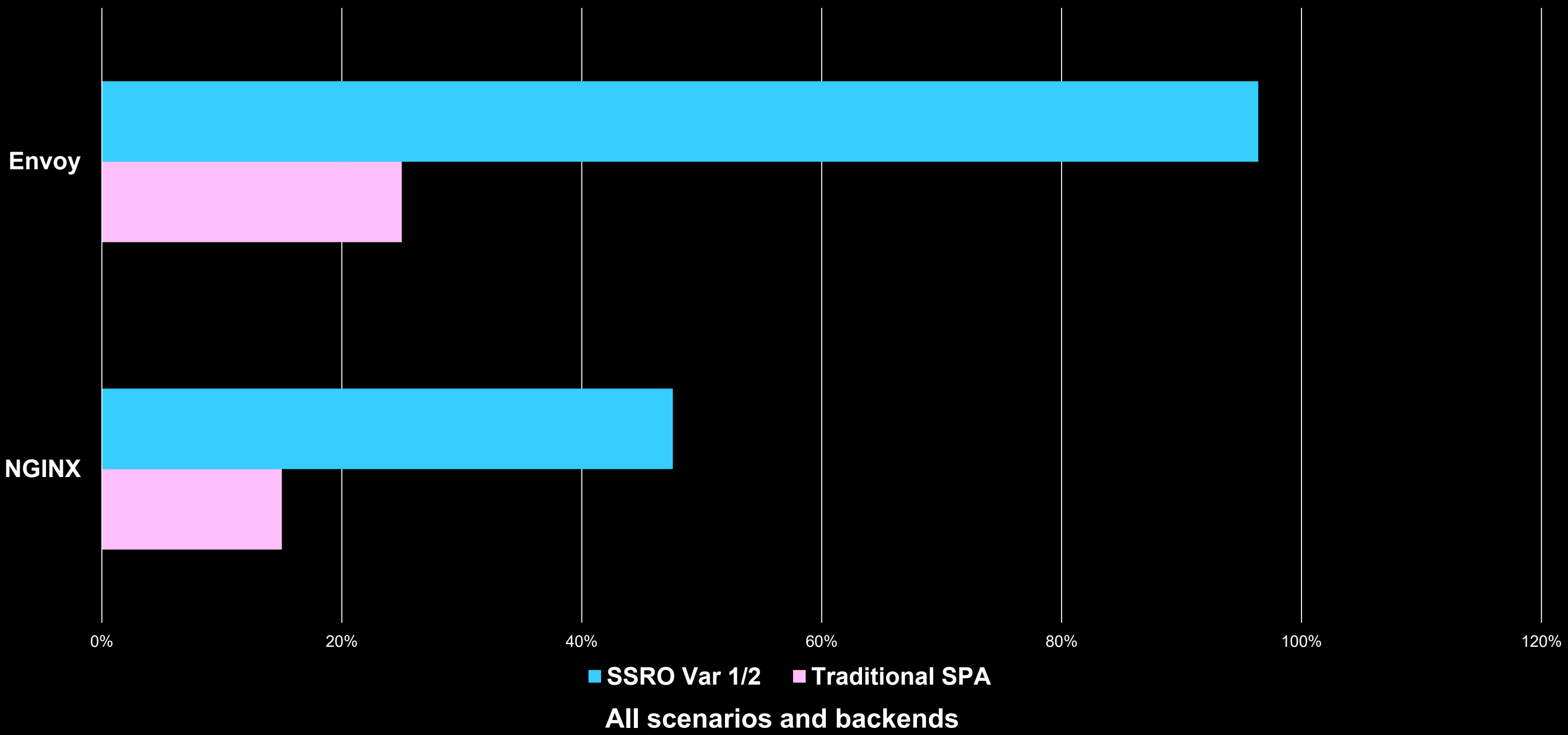


Server-Side Race Orchestration (SSRO)

- **QPACK Blocked Streams:** We withhold the stream type identifier byte (0x02) to lock up to 100 parallel streams in an in-memory HoL state. Releasing this byte instantly flushes them simultaneously.
- **Dynamic Table Saturation:** By filling the QPACK compression table with high-entropy, non-indexed headers, we force intensive proxy RAM-buffering, completely flattening network-layer noise.
- **Cross-Protocol Sync:** We utilize a parallel HTTP/2 & HTTP/3 SPA connections as a burst.
- **JSON Padding Latency:** Supplementing transport gates with extensive CRLF padding to force asynchronous back-end web parsers into processing loops, widening the exploitable timing window.
- **QPACK + Priority:** Combining HoL blocking (QPACK Blocked Streams) with u=0 flags for near-zero internal dispatch latency.

The Proxy Trap: Why SSRO Defeats Proxy Buffering

Proxy Buffering vs. Race Success Rates



Multiplying Exploit Windows with SSRO

Attack Variant	Proxy	Backend	Exploitation Class	Empirical Yield	Predictability Metrics (CV)*
Var 1: QPACK HoL Block	Envoy	Go, .NET, FastAPI	Double Spend	96.4% Success	0.31 (High)
Var 2: Dynamic Table Saturation	NGINX	Spring, LSPHP	State Inversion	90.0% Success	0.03 (Perfect)
Var 2: Dynamic Table Saturation	NGINX	Spring, LSPHP	Double Spend	Experimental Peak	0.21 (Superior)
Var 3: Cross-Protocol	HAProxy	All Runtimes	Limit Overrun	16.4x Multiplier	0.42 (Moderate)
Var 4: JSON Padding	Asynchronous Logic Bypass	Go, .NET, FastAPI	Payload-Induced Latency	Latency Pipeline Multiplier	N/A (Varies per target)
Var 5: QPACK Block + RFC 9218 Priority	HAProxy	Go, FastAPI	Double Spend	95.0% Success	0.28 (High)

Multiplying Exploit Windows with SSRO

Stack	DB	# JSON Success	JSON (ms)	# JSON Success	JSON (ms)	EEM
		SPA H2		SSRO var4		
Spring	MySQL	10	225	11	231	1.1x
Spring	Postgres	10	230	12	221	1.2x
Go	MySQL	1	290	10	294	10.0x
Go	Postgres	1	298	10	295	10.0x
LSPHP	MySQL	2	248	0	242	0.0x
LSPHP	Postgres	2	325	6	248	3.0x
.NET	MySQL	20	231	83	228	4.2x
.NET	Postgres	1	623	1	319	1.0x
FastAPI	MySQL	15	251	20	287	1.3x
FastAPI	Postgres	1	488	20	260	20.0x

Multiplying Exploit Windows with SSRO

Stack	DB	# JSON Success	JSON (ms)	# JSON Success	JSON (ms)	EEM
		SPA H3		SSRO var4		
Spring	MySQL	11	99	11	86	1.0x
Spring	Postgres	9	320	11	90	1.2x
Go	MySQL	1	35	10	104	10.0x
Go	Postgres	1	140	10	106	10.0x
LSPHP	MySQL	8	202	8	175	1.0x
LSPHP	Postgres	2	549	2	429	1.0x
.NET	MySQL	5	90	31	85	6.2x
.NET	Postgres	1	647	1	320	1.0x
FastAPI	MySQL	20	134	20	118	1.0x
FastAPI	Postgres	1	383	20	149	20.0x

The Legacy Backend Illusion (H3 → H1.1 Translation)

The Transport Immunity Myth

Developers frequently assume that because their inner application runtimes are locked to legacy **HTTP/1.1** and lack native HTTP/3 multiplexing capabilities, they are structurally immune to modern protocol timing attacks. They falsely rely on upstream protocol translation as a security boundary.

The Memory-to-Socket Railgun

The edge proxy acts as an inadvertent attack amplifier by acting as a **network jitter eraser**. It absorbs fragmented, high-jitter wide-area network (WAN) HTTP/3 streams, reorganizes them cleanly inside its local RAM using QPACK/Priority primitives, and instantly maps the unblocked payloads onto parallel upstream HTTP/1.1 TCP connections over an ultra-low-latency cloud LAN.

Systemic Target Expansion

This architectural gap completely shatters traditional threat modeling scope. The vulnerability is no longer restricted to bleeding-edge, end-to-end HTTP/3 stacks—**every legacy enterprise backend deployment sitting behind a modern, H3-enabled load balancer or reverse proxy is critically exposed.**

Introducing TimeOrch & Live Demo

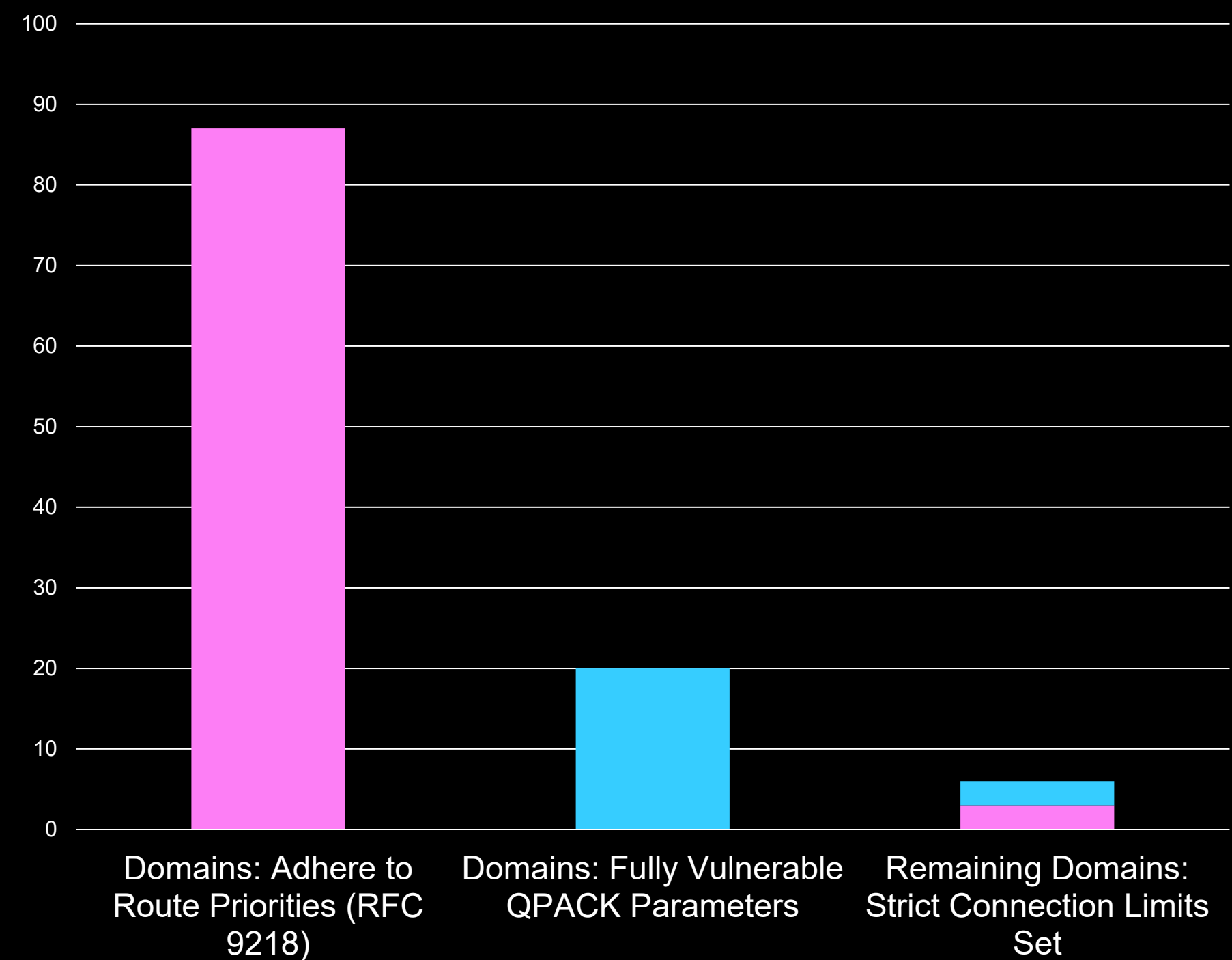
TimeOrch: A novel, open-source automation framework designed for advanced HTTP/3 and RFC 9218 orchestration.

Live Demo Scenario:

- Targeting an enterprise application protected by an aggressive edge proxy architecture.
- Executing SSRO Var 2 (Dynamic Table Saturation).
- Visual Check: Watch the network traffic visibly pause as requests freeze in the proxy's memory, followed by a zero-jitter burst that executes a massive double-spend attack.

Live Demo

Global Scanning Data & The Vendor Coordination Breakdown



Coordinated disclosure via CERT/CC was met with pushback. Vendors dismissed stream prioritization flaws as "working as intended," creating an unpatched protocol-layer vulnerability.

Multi-Layer Remediation Strategy

Proxy Hardening

- Force recompilation of edge proxies to strictly set:
 - QPACK_MAX_TABLE_CAPACITY = 0 and
 - QPACK_BLOCKED_STREAMS = 0, neutralizing timing alignment opportunities.

Application Framework Realities:

- Asynchronous event loops (.NET, Go, FastAPI) fundamentally facilitate these attacks due to their non-blocking execution models.
- Thread-per-process architectures (like legacy LSPHP) offer higher natural resistance but are highly inefficient.

The 67% Database Resilience Variance:

- MySQL (Failed): Permitted up to 16.4x drains due to weaker default concurrency controls.
- PostgreSQL (Succeeded): Restrained violations to 1.0x due to its strict MVCC implementation.

Mandatory Requirements:

- Implement explicit pessimistic row-level locks (SELECT FOR UPDATE).
- Enforce SERIALIZABLE isolation levels.
- Warning: Distributed environments must use true distributed locks (Redlock); native single-threaded Redis execution is not a safety guarantee against SSRO.

Conclusion & Q&A

- **The Shift:** Race conditions are now deterministic, server-side protocol attacks.
- **The Weapon:** Weaponizing QPACK (RFC 9204) and RFC 9218 completely neutralizes edge proxy buffering and reordering attacks.
- **The Defense:** Mitigation requires strict proxy tuning and explicit database-layer isolation.
- **TimeOrch:** <https://github.com/efchatz/timeorch>